PENDAHULUAN

Halo teman teman dsc semua di modul ini kita bakal bahas dasar dasar yang ada di javascript, sebenarnya apasih itu javascript?

JavaScript adalah bahasa pemrograman yang digunakan untuk membuat aplikasi menjadi interaktif dan bisa menjalankan berbagai instruksi.
Bayangkan website itu seperti sebuah rumah □

- HTML = kerangka rumah, seperti dinding dan pintu.
- CSS = dekorasi rumah, seperti warna cat dan desain interior.
- JavaScript = sistem yang membuat rumah bisa melakukan sesuatu, seperti lampu yang menyala ketika sakelar ditekan.

Artinya, JavaScript bisa menyimpan data, memeriksa kondisi, dan menjalankan instruksi sesuai aturan yang kita buat.

Tapi JavaScript bukan cuma buat website! Dengan bantuan Node.js, JavaScript juga bisa digunakan untuk membuat program terminal, backend, API, bahkan aplikasi lainnya.

Gampangnya, JavaScript adalah bahasa untuk memberi tahu komputer apa yang harus dilakukan, kapan melakukannya, dan bagaimana cara melakukannya. □

*s*umber catgipiteh (chat-gpt)*

Persiapan:

- visual studio code

- node js (opsional)

- niat

(kalau mau pakai javascript website boleh, hubungkan sama file html kalian, buka file nya entar menuju ke web, klik kanan halaman web html kosong kalian, klik inspect, lalu console. Di console itu kalian dapat melihat hasil output kalian. ***Run nya gimana bang?*** Refresh aja website nya tiap kalian kelar nulis kode javascriptnya)

PEMBAHASAN

A. variable

Apa itu variabel?

Variabel itu ibarat wadah buat menyimpan data, ngab. Jadi, kita bisa menyimpan informasi dengan nama tertentu, lalu menggunakan informasi itu kapan pun dibutuhkan dalam program.

Contoh kita punya kode disini:

latihan1.js > ...

```
1   //variable
2   const nama = "arya" //tipe data string
3   const buah = "pisang" //tipe data string
4   const angka = 1 //tipe data int/angka bulat
5   let boolean = true //tipe data boolean atau benar salah (true false)
6
7   //output biasa
8   console.log(nama)
9
10  //ouput gabungan
11  console.log("nama saya adalah " + nama + " buah favorit saya " + buah)
12
13  //output gabungan dengan template strings
14  console.log(`nama saya adalah ${nama} saya suka ${buah} dan itu ${boolean}`)
15
```

Setiap variabel punya nama dan menyimpan nilai yang berbeda, ngab. Dengan begitu, kita nggak perlu menulis ulang nilainya terus-menerus.

Terus, apa bedanya const dan let?

1. const = variabel yang tidak bisa diberi nilai baru melalui assignment atau constant (tidak boleh diubah).
2. let = variabel yang nilainya bisa diubah.

Contohnya:

```
const nama = "arya";

let buah = "pisang";


buah = "apel"; // Bisa, ngab!


// nama = "budi"; // Error kalau dijalankan kalau nda percaya coba aja terus baca error nya

console.log(buah); // apel
```

Jadi, kalau datanya memang nggak perlu diganti, biasanya gunakan const. Kalau perlu diubah selama program berjalan, gunakan let.

1. output biasa

```
//output biasa
console.log(nama)
```

Cara kerjanya: JavaScript mengambil nilai yang tersimpan di variabel nama, lalu menampilkannya.

Metode ini cocok kalau lu cuma mau menampilkan satu nilai, misalnya nama, angka, atau hasil perhitungan.

Output:

```
raihan@fedora:~/Documents/dsr-jvscript$ node latihan1.js
arya
```

2. Ouput gabungan dengan +

```
// //ouput gabungan
console.log("nama saya adalah " + nama + " buah favorit saya " + buah)
```

Cara kerjanya: tanda + menggabungkan beberapa potongan teks dan nilai variabel menjadi satu string.

Output:

```
raihan@fedora:~/Documents/dsr-jvscript$ node latihan1.js
nama saya adalah arya buah favorit saya pisang
```

3. Output gabungan menggunakan template strings

```
// //output gabungan dengan template strings
console.log(`nama saya adalah ${nama} saya suka ${buah} dan itu ${boolean}`)
```

Cara kerjanya: lu menulis teks di antara backtick (`), lalu menyisipkan nilai variabel menggunakan ${...}.

Bahkan, lu bisa memasukkan ekspresi:

console.log(`Hasil 10 + 5 adalah ${10 + 5}`);

Cara ini biasanya lebih enak dibaca ketika lu membuat teks yang mengandung banyak variabel.

Output:

```
raihan@fedora:~/Documents/dsr-jvscript$ node latihan1.js
nama saya adalah arya saya suka pisang dan itu true
```

B. Operator Logika dan perbandingan

Apa itu operator logika?

Operator logika adalah simbol yang digunakan untuk menggabungkan atau membalik suatu kondisi yang hasilnya berupa true (benar) atau false (salah).

Operator logika yang umum digunakan:

- && (AND): kedua kondisi harus benar.
- || (OR): minimal satu kondisi harus benar.
- ! (NOT): membalik nilai logika.

Apa itu operator perbandingan?

Operator perbandingan adalah simbol yang digunakan untuk membandingkan dua nilai, kemudian menghasilkan nilai logika true atau false.

Operator perbandingan yang umum digunakan:

Operator	Arti	Contoh	Hasil
>	Lebih besar	10 > 5	true
<	Lebih kecil	10 < 5	false
>=	Lebih besar atau sama	10 >= 10	true
<=	Lebih kecil atau sama	5 <= 3	false
===	Sama nilai dan tipe	5 === 5	true
!==	Tidak sama nilai atau tipe	5 !== "5"	true

Kenapa kita membutuhkan keduanya?

Karena program tidak hanya menyimpan dan menampilkan data. Program juga harus bisa memeriksa keadaan dan menentukan tindakan berdasarkan keadaan tersebut.

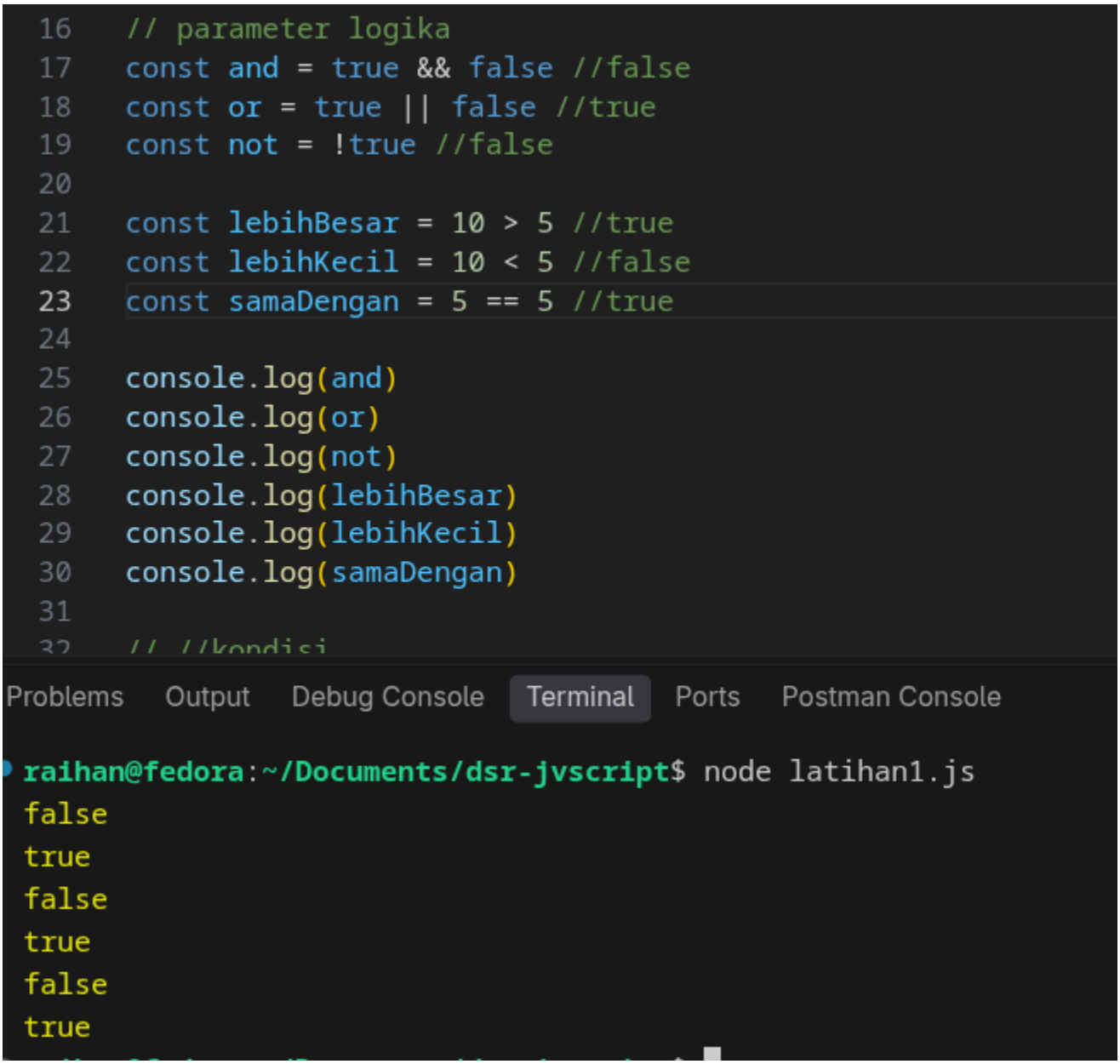
Contoh kode kita:

```
// parameter logika
const and = true && false //false
const or = true || false //true
const not = !true //false

const lebihBesar = 10 > 5 //true
const lebihKecil = 10 < 5 //false
const samaDengan = 5 == 5 //true

console.log(and)
console.log(or)
console.log(not)
console.log(lebihBesar)
console.log(lebihKecil)
console.log(samaDengan)
```

Output:



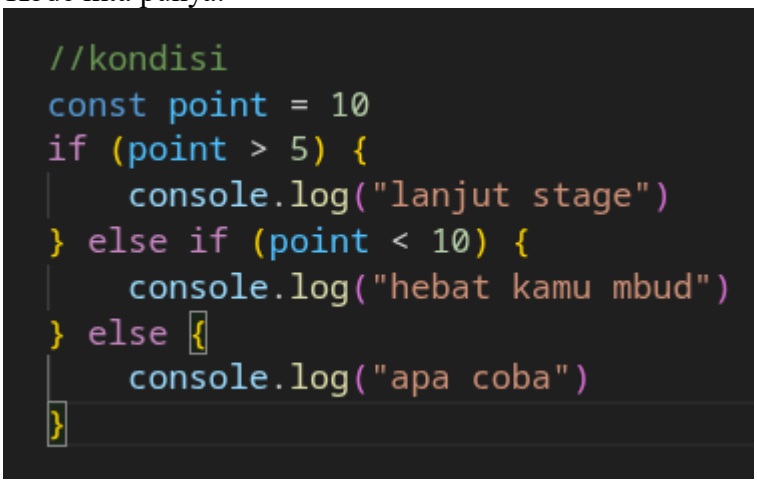
Tes tulis sendiri ya le buktikan sendiri gimana hasilnya, terus kalian modif modif ae dah isi variable nya sendiri biar dapat feel nya, anjay feel.

Intinya, lek: operator perbandingan digunakan untuk memeriksa hubungan antara nilai, sedangkan operator logika digunakan untuk menggabungkan atau membalik kondisi. Keduanya membuat program mampu mengambil keputusan, bukan sekadar menjalankan instruksi secara berurutan.

C. IF statement atau kondisi

if statement atau condition itu digunakan untuk membuat program mengambil keputusan berdasarkan suatu kondisi.

Kode kita punya:



```
//kondisi
const point = 10
if (point > 5) {
    console.log("lanjut stage")
} else if (point < 10) {
    console.log("hebat kamu mbud")
} else {
    console.log("apa coba")
}
```

Kita anggap ini sistem game yang menentukan pesan berdasarkan jumlah poin pemain.

1. `if (point > 5)`

Program memeriksa apakah `point` lebih besar dari `5`.

Karena `point = 10`, kondisi ini bernilai `true`.

Dijalankan

Output: `lanjut stage`

`2. else if (point < 10)`

Bagian ini menjadi kondisi alternatif jika `if` sebelumnya tidak terpenuhi.

Karena kondisi `if` pertama sudah `true`, bagian ini tidak diperiksa lagi.

3. `else`

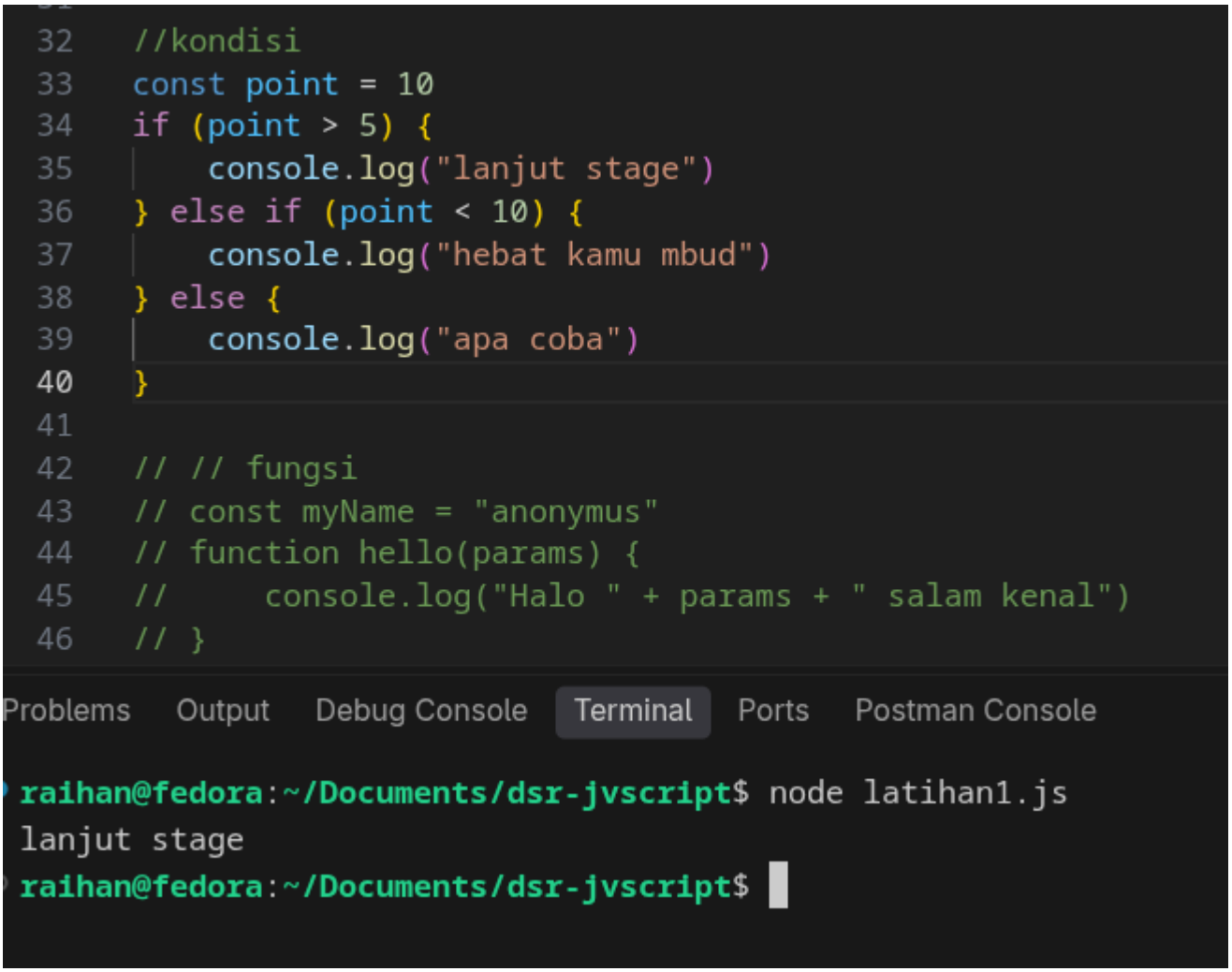
Ini adalah pilihan terakhir ketika semua kondisi sebelumnya tidak terpenuhi.

Karena kondisi pertama sudah terpenuhi, `else` juga dilewati.

Nah, ini konsep penting, ngab: dalam satu rangkaian `if - else if - else`, begitu satu kondisi terpenuhi, blok yang bersangkutan dijalankan dan blok alternatif lainnya dilewati.

Buat testing kondisi else kalian bisa tes masukin data string contoh “admin lengser” (canda min) ke dalam variable point disana, terus kalau mau coba coba kondisi lain atau nambahin kalian bisa coba tambahin sesuai dengan imajinasi kalian aja. Modif modif ae lah itu kodenya.

Output:



Kesimpulan, ngab

- `if` = kondisi pertama yang diperiksa.
- `else if` = kondisi alternatif jika kondisi sebelumnya tidak terpenuhi.
- `else` = pilihan terakhir jika semua kondisi sebelumnya salah.

Satu hal lagi: kalau lu ingin setiap kondisi diperiksa secara independen, gunakan beberapa `if` terpisah, bukan satu rangkaian `if - else if - else`.

D. Function

Opo iku function lek?

function adalah blok kode yang bisa dipanggil ketika dibutuhkan. Pada kode kamu, `hello` adalah nama function, `params` adalah parameter, dan `myName` atau `"Raihan"` adalah argument yang dikirimkan saat function dipanggil.

Contoh kode:

```
// fungsi
const myName = "anonymus"
function hello(params) {
    console.log("Halo " + params + " salam kenal")
}

hello(myName)
hello("Raihan")
```

- `function` → keyword untuk membuat sebuah function.
- `hello` → nama function yang kamu buat.
- `params` → parameter, yaitu variabel yang menerima nilai ketika function dipanggil.
- `{ ... }` → isi function, yaitu kode yang akan dijalankan ketika function dipanggil.

Gampangnya, function itu seperti mesin yang kita buat untuk menjalankan tugas tertentu. Dalam contoh ini, tugasnya menampilkan salam perkenalan.

1. Memanggil function dengan variabel

```
const myName = "anonymus";

hello(myName);
```

Di sini, kamu memanggil function `hello()` dan memberikan variabel `myName` sebagai input.

Karena nilai `myName` adalah `"anonymus"`, maka parameter `params` menerima nilai tersebut.

Hasilnya:

```
Halo anonymus salam kenal
```

2. Memanggil function dengan nilai langsung

```
hello("Raihan");
```

Kali ini kamu tidak menggunakan variabel, tetapi langsung mengirimkan string `"Raihan"` ke function.

Parameter `params` menerima nilai `"Raihan"`, sehingga hasilnya:

```
Halo Raihan salam kenal
```

Output:

```
// fungsi
const myName = "anonymus"
function hello(params) {
    console.log("Halo " + params + " salam kenal")
}

hello(myName)
hello("Raihan")
```

```
Problems  Output  Debug Console  Terminal  Ports  Postman Console

raihan@fedora:~/Documents/dsr-jvscript$ node latihan1.js
Halo anonymus salam kenal
Halo Raihan salam kenal
raihan@fedora:~/Documents/dsr-jvscript$
```

Jadi, satu function yang sama bisa digunakan berkali-kali dengan input berbeda tanpa perlu menulis ulang kode `console.log()`.

Apa bedanya parameter dan argument?

Ini dua istilah yang perlu kamu pahami, ngab.

- Parameter

  `function hello(params) {`

  `params` adalah nama variabel untuk menerima nilai dari pemanggilan function.

- Argument

  `hello("Raihan");`

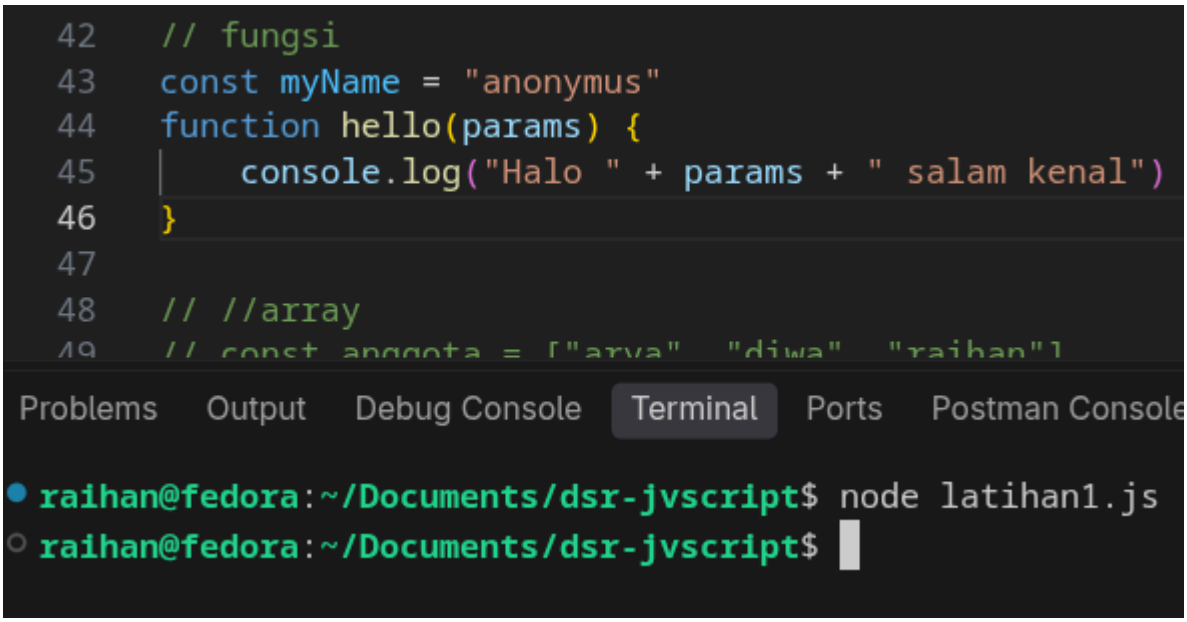
  `"Raihan" adalah nilai yang dikirimkan ketika function dipanggil.`

Analogi sederhananya:

- Parameter = wadah untuk menerima nilai.
- Argument = nilai yang dimasukkan ke wadah tersebut.

Nama `params` sebenarnya bebas. Kamu bisa menggantinya menjadi `nama` agar lebih mudah dibaca.

Bagaimana kalau function tidak dipanggil?



Program tidak akan menampilkan salam apa pun karena kamu baru mendefinisikan function, belum menjalankannya.

Function baru dijalankan ketika dipanggil:

```
hello("Raihan");
```

Ini alasan function berguna: kita bisa mendefinisikan tugas sekali, lalu memanggilnya kapan pun dibutuhkan.

E. Array

Apa itu array?

Array adalah tempat untuk menyimpan beberapa nilai dalam satu variabel.

Misalnya, kamu ingin menyimpan nama anggota UKM. Tanpa array, kamu bisa menulis seperti ini:

```
const anggota1 = "arya";
const anggota2 = "diwa";
const anggota3 = "raihan";
```

Tapi kalau anggotanya ada 50 orang, bakal merepotkan kalau harus membuat 50 variabel, kan?

Nah, dengan array, kamu cukup menulis:

```
const anggota = ["arya", "diwa", "raihan"];
```

Sekarang ketiga nama tersebut tersimpan dalam satu variabel bernama `anggota`.

Contoh kode:

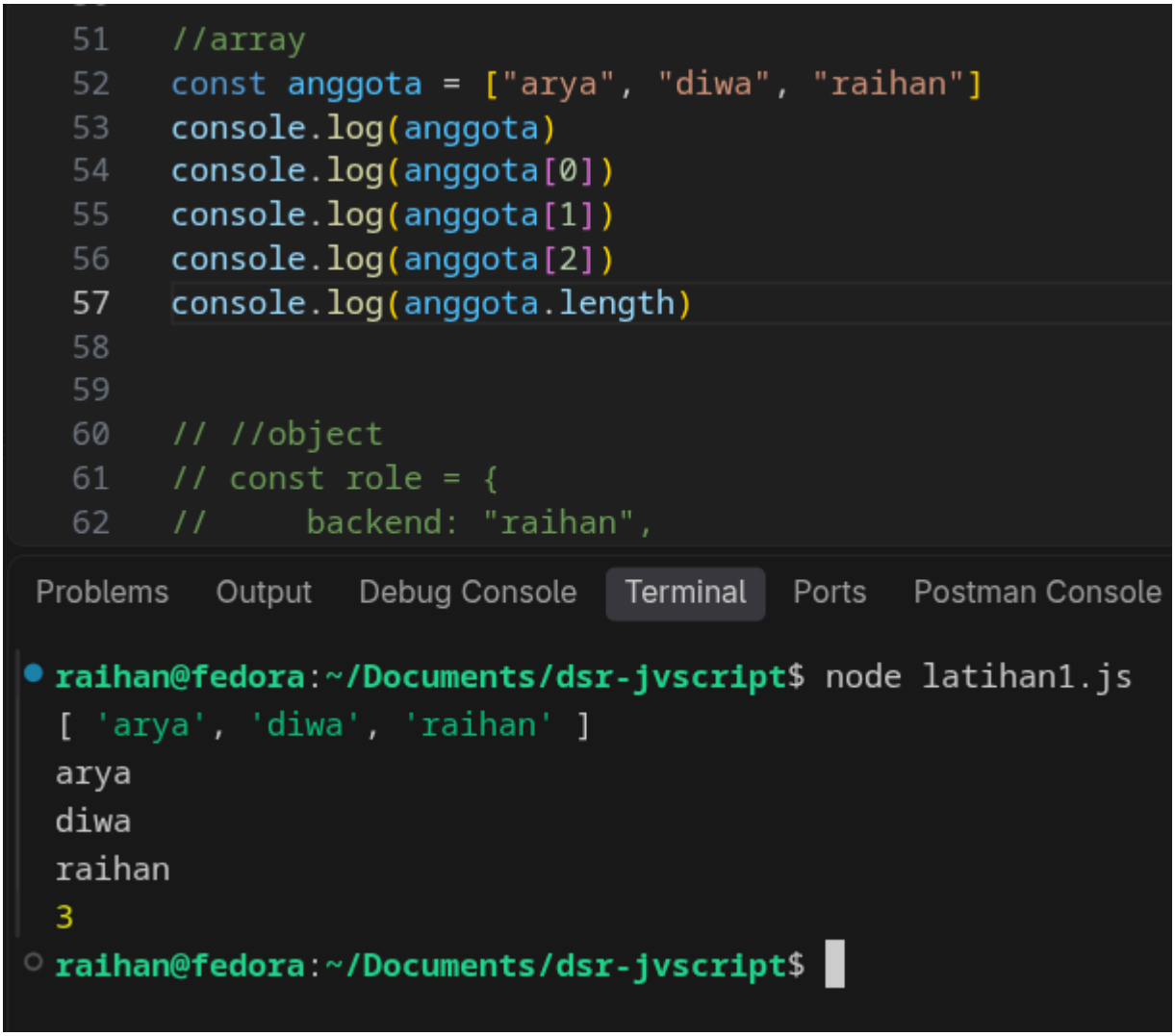
```
//array
const anggota = ["arya", "diwa", "raihan"]
console.log(anggota)
console.log(anggota[0])
console.log(anggota[1])
console.log(anggota[2])
console.log(anggota.length)
```

Kita bedah satu per satu.

- `const anggota` → membuat variabel bernama `anggota`.
- `"arya"`, `"diwa"`, `"raihan"` → nilai atau elemen yang disimpan dalam array.
- `console.log(anggota)` → menampilkan seluruh isi array.
- `anggota[0]` → mengambil elemen pada index 0.
- `anggota[1]` → mengambil elemen pada index 1.
- `anggota[2]` → mengambil elemen pada index 2.

Untuk mengetahui jumlah elemen dalam array, kamu bisa menggunakan `.length`:

Output:



Nomor urutan array dimulai dari 0!

Nah, ini bagian paling penting, ngab. Di JavaScript, array menggunakan sistem index yang dimulai dari angka 0, bukan 1.

Gimana kalau nomor index nya gaada?

Kalian bisa coba sendiri gimana kalau kalian masukin nomor index yang tidak tersedia di dalam array tersebut, contoh console.log(anggota[4]) kalian coba aja apa ouput yang dihasilin sama javascriptnya.

F. Object

Apa itu object?

Object adalah struktur data yang menyimpan informasi dalam bentuk pasangan `key: value`.

Contohnya, kamu ingin menyimpan informasi tentang anggota UKM berdasarkan bidangnya. Nah, object cocok untuk kebutuhan seperti ini.

1. Object doang

kode:

```
//object
const role = {
    backend: "raihan",
    frontend: "arya",
    fullstack: "al dhiwa"
}
console.log(role)
console.log(role.backend)
console.log(role.frontend)
console.log(role.fullstack)
```

KEY	VALUE
backend	“raihan”
frontend	“arya”
fullstack	“al dhiwa”

Penjelasannya:

- `const role` → membuat variabel bernama `role`.
- `{ ... }` → menandakan object.
- `backend`, `frontend`, `fullstack` → disebut key atau nama properti.
- `"raihan"`, `"arya"`, `"al dhiwa"` → disebut value atau nilai properti.
- Tanda `:` memisahkan key dan value.
- Tanda koma `,` memisahkan satu properti dengan properti lainnya.

Bagaimana cara mengambil datanya?

`console.log(role.backend);`
JavaScript mencari properti bernama `backend` di dalam object `role`, lalu menampilkan nilainya.

Output:

`raihan`

Cara ini disebut dot notation atau notasi titik.

Kamu juga bisa menggunakan bracket notation:

console.log(role["backend"]);

Hasilnya sama, yaitu raihan.

```
60  //object
61  const role = {
62      backend: "raihan",
63      frontend: "arya",
64      fullstack: "al dhiwa"
65  }
66  console.log(role)
67  console.log(role.backend)
68  console.log(role.frontend)
69  console.log(role.fullstack)
```

Problems Output Debug Console Terminal Ports Postman Console

```
raihan@fedora:~/Documents/dsr-jvscript$ node latihan1.js
{ backend: 'raihan', frontend: 'arya', fullstack: 'al dhiwa' }
raihan
arya
al dhiwa
raihan@fedora:~/Documents/dsr-jvscript$
```

Jadi, berbeda dengan array yang menggunakan index angka seperti anggota[0], object biasanya diakses menggunakan nama key seperti role.backend.

2. Object yang berisi array

Di sini, setiap value pada object bukan lagi satu string, melainkan sebuah array yang berisi beberapa nama.

```
71  //object dengan array
72  const roles = {
73      backend: ["raihan", "mirza", "fijar"],
74      frontend: ["arya", "adit", "bang hanif"],
75      fullstack: ["bang hanif", "bang rengga"]
76  }
77  console.log(roles)
78  console.log(roles.backend[0])
79  console.log(roles.frontend[2])
80  console.log(roles.fullstack[1])
```

Di sini ada dua struktur data yang digabungkan:

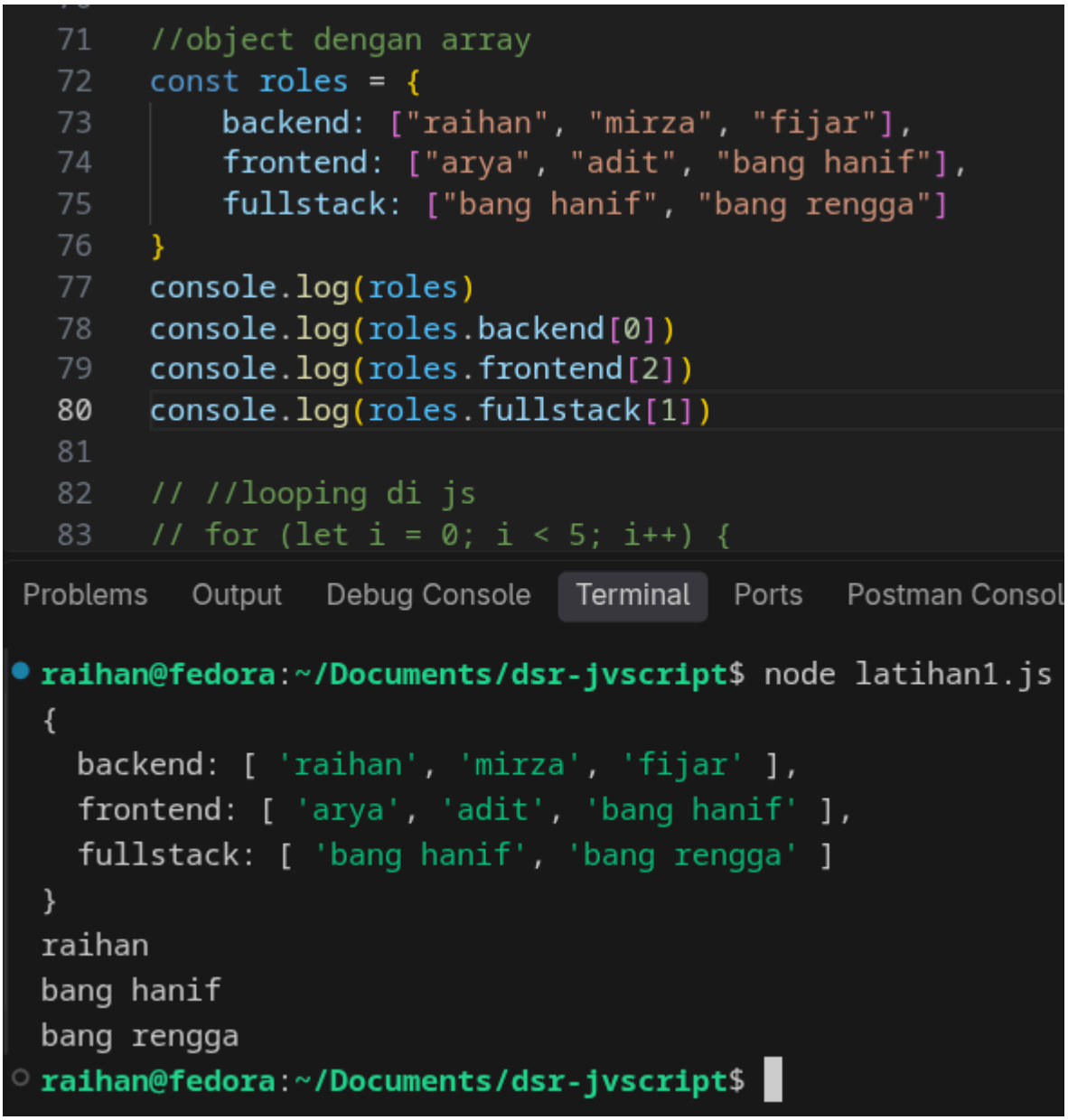
- Object `{}` digunakan untuk mengelompokkan data berdasarkan bidang: `backend`, `frontend`, dan `fullstack`.
- Array `[]` digunakan untuk menyimpan beberapa nama anggota dalam setiap bidang.

Bayangin kita punya struktur anggota ukm dengan dengan 3 keahlian namun dimiliki banyak orang

KEY	VALUE
backend	[“raihan”, “mirza”, “fijar”]
frontend	[“arya”, “adit”, “bang hanif”]
fullstack	[“bang hanif”, “bang rengga”]

Jadi, object `roles` bukan hanya menyimpan nama anggota. Object tersebut mengelompokkan beberapa array berdasarkan bidangnya.

Output:



Bagaimana membaca `roles.backend[0]`?

Bacanya dari kiri ke kanan, ngab:

`roles.backend[0]`

1. `roles` → akses object utama.
2. `.backend` → ambil value dari key `backend`.
3. `[0]` → ambil elemen pertama dari array tersebut.

Kesimpulan: array vs object

Array	Object
Menyimpan kumpulan elemen	Menyimpan pasangan key dan value
Mengakses data dengan index	Mengakses data dengan key
Contoh: `anggota[0]`	Contoh: `role.backend`

Kalau digabungkan, seperti kode `roles` milikmu, kita bisa mengakses data dengan pola object → key → index array.

Konsep ini sering digunakan di JavaScript, terutama ketika membuat data anggota, daftar produk, respons API, dan aplikasi CRUD. Nantinya kamu juga akan sering menemukan array berisi banyak object, misalnya setiap anggota punya `nama`, `nim`, dan `bidang` masing-masing.

G. For Loop

Apa itu `for loop`?

`for loop` adalah perulangan yang digunakan untuk menjalankan kode berkali-kali selama kondisi yang ditentukan masih bernilai `true`.

Contoh kode:

```
82  //looping di js
83  for (let i = 0; i < 5; i++) {
84      console.log("HIDUP DESCEHH!!")
85      console.log(`no = ${i}: AKU DSC`)
86  }
```

Contohnya, kamu ingin menampilkan tulisan `"HIDUP DESCEHH!!"` sebanyak 5 kali. Daripada menulis `console.log()` lima kali, kamu bisa menggunakan loop.

Ada tiga bagian penting di dalam `for`:

```
for (let i = 0; i < 5; i++)
```

1. `let i = 0`

Inisialisasi: membuat variabel `i` dengan nilai awal `0`. Bagian ini dijalankan satu kali sebelum perulangan dimulai.

2. `i < 5`

Kondisi: perulangan berjalan selama nilai `i` kurang dari `5`. Kalau kondisinya `false`, loop berhenti.

3. `i++`

Increment: menambah nilai `i` sebanyak `1` setelah satu putaran selesai. Contohnya, `0` menjadi `1`, lalu `2`, dan seterusnya.

Bagaimana perulangannya berjalan?

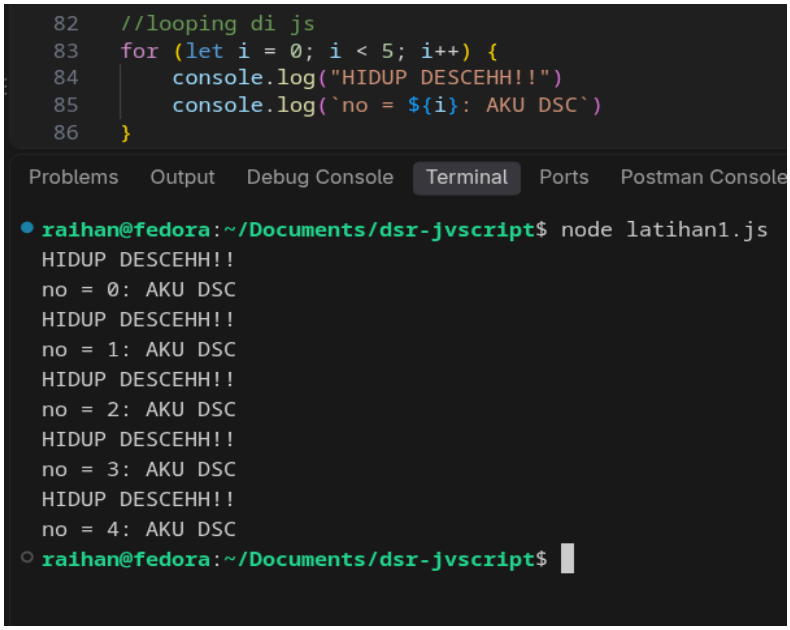
Coba perhatikan urutan berikut.

Putaran	Nilai `i`	`i < 5`	Hasil
1	0	`true`	Kode dijalankan
2	1	`true`	Kode dijalankan
3	2	`true`	Kode dijalankan
4	3	`true`	Kode dijalankan
5	4	`true`	Kode dijalankan
6	5	`false`	Loop berhenti

Kenapa putaran terakhir tidak dijalankan saat `i = 5`? Karena `5 < 5` bernilai `false`.

Jadi, walaupun kondisi menggunakan angka `5`, nilai `i` yang dijalankan adalah 0, 1, 2, 3, dan 4. Totalnya tetap 5 kali.

Output:



Intinya, ngab: `let i = 0` menentukan nilai awal, `i < 5` menentukan kapan loop berhenti, dan `i++` menaikkan nilai penghitung. Kalau tiga bagian ini sudah kamu pahami, kamu sudah menguasai dasar `for loop` di JavaScript!

NGODING DULU KITA LE (latihan)

Kita coba buat sistem crud simpel dulu tanpa database, file, aplikasi pihak ketiga, dan framework apapun, kita bener bener pure gunain array dan object sebagai tempat data kita. Jadi sistem yang mau kita buat adalah sistem CRUD buku.

CRUD mulu jir opo iku CRUD?
CRUD itu singkatan dari (Create, Read, Update, Delete) sistem/fitur ini adalah fondasi awal pada setiap pengembangan aplikasi maupun website berbasis web dinamis (web application).

Disclaimer: mungkin selama kalian koding kode ini kalian akan bertemu dengan fungsi fungsi yang mengurus data array, jika kalian bingung kalian bisa tanya ai aja atau kalian tahan aja rasa penasarannya sampe minggu depan saat materi ini mau dibahas.

Intinya pahami aja dulu dasar dasar javascript ini pelan pelan, lebih baik terlambat daripada tidak sama sekali. Tapi jangan prioritaskan pembelajaran ini jika kalian ada yang lebih penting di hidup kalian, UTS contohnya. Yapping mulu jir noh kodenya:

***scroll bawah**

```
//crud simpel tanpa database ataupun file/aplikasi tambahan
let books = []
let id = 1;

//Create
function createBuku(title, author) {
    const book = {
        id: id,
        title: title,
        author: author
    }

    books.push(book)
    id++
    console.log("Berhasil menambahkan data")
}

//Read
function readBuku() {
    if (books.length === 0) {
        console.log("Belum ada data buku bro")
        return
    }

    for (const book of books) {
        console.log(book)
    }
}

//Update
function updateBuku(id, newTitle) {
    for (const book of books) {
        if(book.id === id) {
            book.title = newTitle
            console.log("Berhasil mengubah data buku")
            return
        }
    }
    console.log("Buku tidak ditemukan")
}

//Delete
function deleteBuku(id) {
    for (let i = 0; i < books.length; i++) {
        if (books[i].id === id) {
            books.splice(i, 1)
            console.log("berhasil menghapus data buku")
            return
        }
    }
    console.log("Buku tidak ditemukan")
}
```

```
140
141  //=== CARA MENGGUNAKAN FUNGSI CRUD YANG SUDAH KITA BUAT ===//
142
143  //Input data kalian ke dalam parameter fungsi yang sudah kita buat sebagai argument
144
145  //create data
146  createBuku("islam ala probroro", "probrooooro")
147  createBuku("kitab asli ngawi", "admin dsc")
148
149  //read data
150  readBuku()
151
152  //update data
153  updateBuku(2, "kitab asli dsc")
154
155  //delete data
156  deleteBuku(2)
157
158  //memanggil fungsi readBuku() untuk membuktikan hasil dari proses crud kita
159  readBuku()
160
```

FYI: Saat melakukan pemanggilan Function saya sarankan kalian coba satu persatu function yang dipanggil agar kalian tau setiap proses bagaimana cara jalannya, saat nulis kode selalu perhatikan detail kecil seperti tanda koma (,), perbedaan antara buka kurung (), kurung kurawal {}, dan kurung siku []. Udah sih itu aja good luck.